Edge cases (a bad release that slips past the gate) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `release_gate_practice.py` (Edge cases section): a gate that can be bypassed just by editing one plain-text file isn't really a gate at all. This section first proves that bypass is genuinely real by actually doing it, then closes it in two stages — first by cryptographically signing the pointer file, then by removing the separate pointer altogether so there's nothing left to edit. **If not:** the Build Task's constraint "the gate must not be skippable by accident" would stay a hope, never something you actually tested and proved.

Every approach below starts by proving the bypass is real, then fixes it a different way.

Solution

Story: a plain hash can be recomputed by anyone who simply reads the source code, which makes it useless as a tamper check. A signature built from a secret key stored only in `.env` genuinely can't be recomputed without that secret. **If not:** a teammate who saw the verification script could just "fix" the check to match their edited pointer, defeating the whole point of it.

```
# release_gate_practice.py — Edge cases section
import hmac
import hashlib
import json
import logging
import os
from dotenv import load_dotenv

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

class MissingSecretError(Exception):
    pass

class TamperedPointerError(Exception):
    pass
```

```
load_dotenv()
SECRET = os.getenv("DEPLOY_GATE_SECRET")
if SECRET is None or SECRET == "":
    raise MissingSecretError(
        "DEPLOY_GATE_SECRET is not set in .env — "
        "pointers can't be signed or checked without it"
    )

def sign(version_name: str) -> str:
    # how: the signature depends on the name AND the secret
    key = SECRET.encode()
    message = version_name.encode()
    return hmac.new(key, message, hashlib.sha256).hexdigest()

def save_pointer(version_name: str) -> None:
    pointer = {"version_name": version_name, "signature": sign(version_name)}
    with open("live_version.json", "w") as f:
        json.dump(pointer, f)

def get_live_version() -> str:
    with open("live_version.json") as f:
        pointer = json.load(f)
    try:
        version_name = pointer["version_name"]
        stored_signature = pointer["signature"]
    except KeyError as e:
        logger.critical(f"Live pointer is missing the field {e}")
        raise TamperedPointerError(
            f"live_version.json has no {e} field — refusing to trust it"
        )

    if not hmac.compare_digest(stored_signature, sign(version_name)):
        logger.critical(
            f"Live pointer signature does not match its version name: "
            f"{version_name}"
        )
        raise TamperedPointerError(
            "live_version.json failed its signature check — "
            "refusing to trust it"
        )
    return version_name

save_pointer("v2")
print(get_live_version())
```

Expected output:

```
v2
```

Hand-edit `live_version.json`'s `"version_name"` to `"v1"` without recomputing `"signature"`:

```
print(get_live_version())
```

Expected output:

```
Live pointer signature does not match its version name: v1
Traceback (most recent call last):
  ...
__main__.TamperedPointerError: live_version.json failed its signature check —
refusing to trust it
```

(the first line is the `CRITICAL` log — this handler has no formatter, so only the message shows). If someone deletes the `"signature"` field instead, the `KeyError` is caught and you get a `TamperedPointerError` naming the missing field, not a bare `KeyError: 'signature'`.

Unlike Basic, an attacker who has read this script still can't produce a matching `"signature"` for `"v1"` without also knowing `SECRET` — `hmac.new` ties the signature to both the version name *and* a value that isn't written down inside `live_version.json` itself.

Revision from Doc01 (Intermediate): two habits come back. First, `load_dotenv()` plus the "required value is missing → raise a named error" rule: the secret comes from `.env`, and if it isn't there, the script stops at startup with `MissingSecretError`. A default secret written in the code would defeat the whole idea — anyone reading the code would know it. Second, catching one *specific* exception, `KeyError`, and raising a named `TamperedPointerError` with a clear message: a pointer with a deleted field is a tampered pointer too, and it should be reported as one, through a named logger at `CRITICAL`.